Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Ders anlatımı İngilizce yapıldı.
  - Asıl anlatım yaklaşık 00:05:30'da başlar.
  - Araya iki ara verildi (yaklaşık 00:58:00-01:10:40 ve 01:55:20-02:06:30).
- Aşağı itmeli otomat (PDA, pushdown automaton)
  - Durumlar, giriş şeridi ve yeni olarak bir yığın (stack).
  - Geçişlerin üç parçası: okunan giriş sembolü, yığından çekilen (pop) sembol, yığına itilen (push) sembol.
  - ε (null) kullanımı ve PDA'nın deterministik olmaması.
- Anlık betimleme (ID, instantaneous description): $(q, \text{kalan giriş}, \text{yığın})$.
- Örnek PDA'lar:
  - M_1: $\{a^n b^n \mid n \geq 0\}$.
  - M_2: $\{vv^R \mid v \in \{a,b\}^*\}$.
  - M_3: a ve b sayısı eşit olan dizgiler.
- Hesaplama ağacı ile $aaabbb$ ve $abba$ girişlerinin adım adım işlenmesi.
- Kabul koşulu: bütün girişin tüketilmesi ve kabul durumunda bitirmek. Yığının içeriği önemli değil.
- Kısaltma: yığından dizgi çekme ve yığına dizgi itme.
- PDA'nın biçimsel tanımı (7'li), hesaplama, $\vdash^*$ gösterimi ve PDA'nın dili.
- Teorem: Bağlamdan bağımsız diller ile PDA'ların tanıdığı diller aynıdır. İspat (dönüştürmeler) bir sonraki derse bırakıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu derste yığının dibinde başlangıçta özel bir sembol vardır: \$.
  - PDA'lar, \$ yığının tepesine geldiğinde yığının sonuna ulaşıldığını anlayacak biçimde tasarlanır.
  - Hoca \$'ın yalnız hesaplamanın sonunda kullanılması gerektiğini söyledi. Bu sözün anlamı için "Ders sonrası notlar"a bakın.
- Boş yığından çekme yapmaya çalışan PDA durur ve girişi reddeder.
  - PDA'larınızı bu durum hiç oluşmayacak biçimde tasarlayın.
- PDA'lar deterministik değildir.
  - Aynı (giriş, çekilen) çifti için birden fazla geçiş olabilir. Giriş okumadan (ε ile) geçiş yapılabilir.
  - Böyle bir noktada makine kendini çoğaltır. Paralel hesaplamalardan biri kabul ederse PDA girişi kabul eder.
- Bu dersin kabul kuralı: Giriş tamamen tüketilmeli ve son durum bir kabul durumu olmalı. Yığında ne kaldığı önemli değil.
  - Bazı kaynaklar yığının boşalmasını da şart koşar. Okuduğunuz kaynağın kuralına dikkat edin.
- PDA'lar iki değeri eşleştirebilir (ör. a ve b sayısı).
  - Hoca "üç ya da daha fazla değeri eşleştiremez" cümlesini yazdı, sonra emin olmadığı için sildi. Yine de genel olarak PDA'nın iki değer eşleştirebildiğini düşündüğünü söyledi (00:41:02).
- Bütün hesaplama ağacını her zaman yazmanız gerekmez. Ama makinenin nasıl çalıştığını anlamak için yararlıdır.

Kısa Tekrar Notları

- **Geçiş etiketi:** $a, b \to c$: girişten a oku, yığının tepesindeki b'yi çek, yığına c it.
  - Herhangi biri ε olabilir: ε giriş okuma yok, ε çekme yok, ε itme yok demektir.
- **ID:** (q, u, s) = (durum, okunmamış giriş, yığın içeriği; tepe solda).
- **Hesaplama:** ID'lerin $\vdash$ ("yields") ile sıralanması. $\vdash^*$: sıfır ya da daha fazla adım.
- **PDA'nın dili:** $L(M) = \{w \mid (q_0, w, Z) \vdash^* (q_f, \varepsilon, s),\ q_f \in F\}$.
- **Biçimsel tanım (7'li):** $M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$.
  - Γ: yığın alfabesi, Z: yığın başlangıç sembolü (\$).
  - Sipser'ın kitabında Z yoktur, tanım 6'lıdır.
- δ **küme döndürür:** $\delta(q_1, a, w_1) = \{(q_2, w_2), (q_3, w_3)\}$.
- **Dizgi çekme/itme kısaltması:** $a, eb \to cdf$: tepedeki e ve altındaki b çekilir, sonra f, d, c sırasıyla itilir ve c tepede kalır.
- **Teorem:** CFL = PDA'ların tanıdığı diller. İspat iki yönlü: her CFG bir PDA'ya, her PDA bir CFG'ye dönüştürülür.

Detaylı Açıklamalar

PDA'nın yapısı

PDA, daha önce görülen otomatlara benzer: durumlar, geçişler, bir başlangıç durumu, kabul ve ret durumları ve giriş dizgisi vardır. Yeni olan parça **yığındır**. Yığın son giren ilk çıkar (LIFO, last in first out) yapısındadır. Yığın kafası her zaman en son eklenen sembolü gösterir.

Geçişler artık yığını da içerir. q_1'den q_2'ye $a, b \to c$ etiketli bir geçiş şunları ister:

- Girişte sıradaki sembol a olmalı.
- Yığının tepesinde b olmalı.

Geçiş olunca a okunur, b çekilir, c itilir.

Örnek: Yığında (tepeden) $B, A, \$$ varken $a, \varepsilon \to \varepsilon$ geçişi olursa yığın değişmez.

Deterministik olmama ve makinenin çoğalması

İki durum PDA'yı deterministik olmayan yapar:

- Aynı parametrelerle ($a, b \to c$) farklı durumlara giden birden fazla geçiş.
- Giriş okumadan yapılan geçiş (ε, ...).

Böyle bir geçiş olunca makine kendini çoğaltır. Örnek: $(q_1, bcacc, A\$)$ ID'sinde $\varepsilon, \varepsilon \to C$ ile q_2'ye geçiş varsa, ikinci makine $(q_2, bcacc, CA\$)$ olur. İki makine paralel çalışır.

Örnek M_1: $\{a^n b^n \mid n \geq 0\}$

Tahtadaki makine:

Geçiş	Etiket	Anlamı
$q_0 \to q_1$	$\varepsilon, \varepsilon \to \varepsilon$	Hiçbir şey yapmadan geç
$q_1 \to q_1$	$a, \varepsilon \to a$	a'ları yığına it
$q_1 \to q_2$	$b, a \to \varepsilon$	İlk b ile eşleştirmeye başla
$q_2 \to q_2$	$b, a \to \varepsilon$	Her b için bir a çek
$q_2 \to q_3$	$\varepsilon, \$ \to \$$	Yığın boşaldıysa (yalnız \$) kabule geç

Kabul durumları q_0 ve q_3'tür. q_0'ın kabul durumu olması $n = 0$ (boş dizgi) içindir.

$aaabbb$ girişinin hesaplaması:

Zaman	ID
0	$(q_0, aaabbb, \$)$
1	$(q_1, aaabbb, \$)$
2	$(q_1, aabbb, a\$)$
3	$(q_1, abbb, aa\$)$
4	$(q_1, bbb, aaa\$)$
5	$(q_2, bb, aa\$)$
6	$(q_2, b, a\$)$
7	$(q_2, \varepsilon, \$)$
8	$(q_3, \varepsilon, \$)$ (çoğalan makine)

7. adımda q_2'deki makine durur ve reddeder (q_2 kabul durumu değildir). $\varepsilon, \$ \to \$$ geçişiyle çoğalan makine q_3'te durur ve kabul eder. Bu yüzden M_1, $aaabbb$'yi kabul eder.

Örnek M_2: $\{vv^R \mid v \in \{a, b\}^*\}$

Geçiş	Etiket
$q_0 \to q_0$	$a, \varepsilon \to a$ ve $b, \varepsilon \to b$ (v'yi yığına it)
$q_0 \to q_1$	$\varepsilon, \varepsilon \to \varepsilon$ (ortayı tahmin et)
$q_1 \to q_1$	$a, a \to \varepsilon$ ve $b, b \to \varepsilon$ (v^R'yi yığınla eşleştir)
$q_1 \to q_2$	$\varepsilon, \$ \to \$$ (kabul)

q_0'daki döngü v'yi yığına iter. q_1'deki döngü girişteki v^R'yi yığından çekilenle karşılaştırır. Ortanın neresi olduğunu makine deterministik olmayan biçimde "tahmin eder": her adımda ε geçişiyle bir kopya q_1'e geçer.

$abba$ girişinde hoca hesaplama ağacını çizdi:

- Her adımda q_0'daki makine bir sembol iter ve aynı zamanda q_1'e geçen bir kopya üretir.
- Yanlış yerde ortayı tahmin eden kopyalar ölür. Örneğin yığının tepesi a iken girişte b okunamaz.
- Girişin sonunda dört hesaplama kalır. Yalnız $(q_2, \varepsilon, \$)$'de olan hesaplama kabul durumundadır. Bu yüzden M_2, $abba$'yı kabul eder.

Kısaltma: dizgi çekme ve itme

Tahtadaki örnek: $q_1 \xrightarrow{a,\ eb \to cdf} q_2$.

- Yığının tepesinde e, altında b olmalı. Bunlar çekilir.
- Yerine cdf itilir. Son durumda c tepededir.

Bu yalnız bir kısaltmadır. Makine aslında sembol sembol çalışır. Olası bir açılım (ara durumlar adsız):

$$q_1 \xrightarrow{a,\ e \to \varepsilon} \cdot \xrightarrow{\varepsilon,\ b \to \varepsilon} \cdot \xrightarrow{\varepsilon,\ \varepsilon \to f} \cdot \xrightarrow{\varepsilon,\ \varepsilon \to d} \cdot \xrightarrow{\varepsilon,\ \varepsilon \to c} q_2$$

Örnek M_3: a ve b sayısı eşit olan dizgiler

q_1'de altı geçişli bir döngü ve $q_1 \to q_2$ için $\varepsilon, \$ \to \$$ vardır. q_2 kabul durumudur.

Okunan	Yığın tepesi	Yapılan	Anlamı
a	\$	$\$ \to 0\$$	İlk fazla a
a	0	$0 \to 00$	Fazla a sayısını artır
a	1	$1 \to \varepsilon$	Bir fazla b'yi kapat

Okunan	Yığın tepesi	Yapılan	Anlamı
b	\$	$\$ \to 1\$$	İlk fazla b
b	1	$1 \to 11$	Fazla b sayısını artır
b	0	$0 \to \varepsilon$	Bir fazla a'yı kapat

Yığın, o ana kadar okunan dizgide hangi harfin kaç tane fazla olduğunu tutar. M_1'den farkı: sıraya bakmaz, yalnız sayıların eşitliğine bakar. Hoca bu örneği evde $abbbaa$ gibi girişlerle denemenizi istedi.

Biçimsel tanım ve PDA'nın dili

- $M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$.
  - Q: durumlar. Σ: giriş alfabesi. Γ: yığın alfabesi. Giriş ve yığın alfabeleri ayrıdır. Girişte a okuyup yığına X itebilirsiniz.
  - δ: geçiş fonksiyonu. Sonuçları kümedir, çünkü PDA deterministik değildir.
  - q_0: başlangıç durumu. Z: yığın başlangıç sembolü. F: kabul durumları.
- ID örneği: $(q_1, aab, ab\$)$ = q_1'deyiz, okunmamış giriş aab, yığında tepeden $a, b, \$$.
- Hesaplama (hocanın genel bir örnek olarak yazdığı satır, M_1'in izlemesi değildir; M_1'de $(q_1, aaabbb, a\$)$ oluşmaz): $(q_0, aaabbb, \$) \vdash (q_1, aaabbb, a\$) \vdash \cdots \vdash (q_3, \varepsilon, \$)$.
  - Kısaca: $(q_0, aaabbb, \$) \vdash^* (q_3, \varepsilon, \$)$.
- Dil: $L(M) = \{w \mid (q_0, w, Z) \vdash^* (q_f, \varepsilon, s),\ q_f \text{ kabul durumu}\}$. Giriş kısmının ε olması şarttır. Yığın içeriği s herhangi olabilir.
- Örnek: $n \geq 1$ için $(q_0, a^n b^n, \$) \vdash^* (q_3, \varepsilon, \$)$. $n = 0$ için boş dizgi q_0'da kalarak kabul edilir. Bu, yalnız $a^n b^n$ dizgilerinin dilde olduğunu gösterir. Başka hiçbir dizginin kabul edilmediği de gösterilirse $L(M_1) = \{a^n b^n \mid n \geq 0\}$ olur (bkz. ders sonrası notlar).

CFL ve PDA denkliği

- Teorem: Bağlamdan bağımsız gramerlerin ürettiği diller, PDA'ların tanıdığı dillerle aynıdır.
- İspat için iki şey gösterilir:
  - Her CFG G için $L(G) = L(M)$ olan bir PDA M vardır.
  - Her PDA M için $L(M) = L(G)$ olan bir CFG G vardır.
- Dönüştürmeler bir sonraki derste anlatılacak.

Ders sonrası notlar

- **$\vdash$ sembolünün okunuşu:** Hoca bu sembolü "yields" diye okuduğunu ama emin olmadığını söyledi. Doğrudur. Sembolün adı turnstile'dır, "yields" ya da "verir" diye okunur. Kitaplarda $\vdash$ olarak geçer.
- **"Giriş ε olabiliyor, demek ki PDA deterministik değil" çıkarımı:** ε-geçişi tek başına deterministik olmamayı gerektirmez. Deterministik PDA'larda (DPDA) da ε-geçişleri olabilir; yeter ki aynı durumda ve aynı yığın tepesinde başka bir geçişle çakışmasın. PDA'yı deterministik olmayan yapan şey, bir anda birden fazla geçişin uygulanabilmesidir.
- **Deterministik olmamanın önemi:** DFA ile NFA aynı güçtedir, ama PDA'larda durum farklıdır. $\{vv^R\}$ gibi bazı bağlamdan bağımsız dilleri deterministik bir PDA tanıyamaz. Ortayı "tahmin etmek" için deterministik olmama gerekir.
- **"PDA'lar yalnız iki değeri eşleştirebilir" sözü:** Hocanın sildiği cümle ve ardından söylediği genel kanı bu kaba hâliyle doğru değildir. Örneğin $\{a^n b^n c^m d^m\}$ ya da $\{a^n b^m c^m d^n\}$ bağlamdan bağımsızdır ve dört blok içerir. Doğrusu: İç içe ya da ardışık eşleşmeler yapılabilir, ama $\{a^n b^n c^n\}$ gibi üç sayının birlikte eşit olması gereken diller bağlamdan bağımsız değildir.
- $aaabbb$ **izlemesi:** Tahtadaki izlemede 4. adımda yığına bir a yazılması unutuldu. Hoca bunu fark edip düzeltti. Yukarıdaki tablo düzeltilmiş hâlidir.
- $abba$ **izlemesi:** Hoca $q_1 \to q_2$ geçişini bir an yığında \$ dışında sembol varken uyguladı, sonra "bu geçiş boş yığın ister" diyerek sildi. Doğrusu: $\varepsilon, \$ \to \$$ yalnız yığının tepesi \$ iken uygulanabilir.

- **$'ın kullanımı:** Hocanın "yalnız sonda" sözü lafzen alınmamalıdır. Anlamı, yığının dibine dokunulmaması ve yığının hiç boşalmamasıdır. M_3'te $a, \$ \to 0\$$ ve $b, \$ \to 1\$$ geçişleri hesaplamanın ortasında da $'ı çeker, ama hemen geri iter.
- $L(M_1)$ **çıkarımı:** Hoca $(q_0, a^n b^n, \$) \vdash^* (q_3, \varepsilon, \$)$ bağıntısını $n \geq 0$ için söyledi ve dilin kalıba eşit olduğunu çıkardı. Bağıntı yalnız $n \geq 1$ için geçerlidir. Ters yön de gösterilmelidir: a'lar yalnız q_1'de okunur, b'ler başladıktan sonra a okunamaz, b sayısı a sayısını geçemez, q_3'e yalnız yığında $ kalmışken geçilir.
- $ **ve yığın alfabesi:** $ yığına konduğu için Γ'nın bir elemanıdır. Bazı kitaplarda $, ilk geçişte ($\varepsilon, \varepsilon \to \$$) yığına itilir. Bu derste $ en baştan yığında kabul edilir.
- **Doğrulama:** M_1, M_2 ve M_3 Python ile simüle edildi. 8 sembole kadar bütün $\{a, b\}$ dizgilerinde sırasıyla $\{a^n b^n\}$, çift uzunluklu palindromlar ve a sayısı b sayısına eşit dizgiler dilini tanıdıkları doğrulandı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz. Ders İngilizce anlatılmıştır; altyazı da büyük ölçüde İngilizcedir.